

软件开发实践一立项报告

一、题目

面向大学生求职的岗位数据分析与个性化推荐系统

二、摘要

本项目面向实习和校招阶段的大学生。针对岗位信息分散、技能要求难比较、学生难以判断岗位匹配度的问题，系统采集、清洗并分析公开岗位数据，提供岗位检索、技能统计、薪资分析、岗位聚类、薪资预测和个性化推荐。项目采用 Python、pandas、scikit-learn 和 Streamlit，使用 KMeans、随机森林回归、TF-IDF 与余弦相似度完成分析，并用图表展示结果。

三、调研

1. 同类产品比较

调研对象	主要功能	技术和数据特点	与本项目的差异
BOSS直聘	职位搜索、公司查看、校园招聘、求职者与招聘方沟通	按职位、城市、行业等条件组织岗位信息，平台内部推荐和匹配算法未公开	本项目不做聊天和投递，重点分析岗位数据，展示技能、薪资和推荐理由
智联校园	校招、实习、专业筛选、职位搜索、简历投递	面向学生提供专业、职位和校园招聘等筛选入口，具体推荐算法未公开	本项目增加岗位聚类、薪资预测和跨岗位统计，帮助学生比较不同方向
猎聘	全职、实习、校招、职位搜索和职业发展服务	公开页面包含岗位搜索和推荐等功能，具体推荐算法和训练数据未公开	本项目使用公开数据和传统机器学习方法，模型输入、指标和推荐依据可以直接查看

2. 调研评述

三个平台都能完成岗位搜索，也提供一定程度的岗位推荐。但平台内部的数据和推荐算法通常不公开，学生能够看到推荐岗位，却不一定能看到排序依据。平台主要服务于真实求职流程，岗位数据也不适合直接用于课堂上的清洗、算法对比和可视化实验。

本项目将范围控制在学生求职分析和岗位推荐，不做招聘平台的完整替代。系统使用公开岗位数据，采用 KMeans、随机森林回归和 TF-IDF 相似度三种可以解释的算法，展示岗位群组、薪资预测结果、匹配技能和缺失技能。这样既保留了招聘场景，也能完成数据处理、算法和可视化功能。

项目主要参考以下公开技术资料和数据：

- [LinkedIn Job Postings Dataset](#)，包含岗位标题、岗位描述、薪资、地点、工作类型、经验等级和技能等字段；
- [scikit-learn 文本特征提取文档](#)，用于 TF-IDF 文本表示；
- [scikit-learn 聚类文档](#)，用于 KMeans 岗位聚类；

- [scikit-learn 集成学习文档](#)，用于随机森林回归。

四、需求列表

采用用户故事描述主要需求。

1. 作为一名准备实习或校招的学生，我希望按照岗位关键词、地区、经验、工作方式和薪资范围筛选岗位，从而快速找到符合条件的岗位。
2. 作为一名学生，我希望查看不同岗位类别、地区和经验等级的岗位数量及薪资分布，从而比较不同求职方向。
3. 作为一名学生，我希望查看岗位中出现频率较高的技能，从而了解目标岗位对技能的要求。
4. 作为一名学生，我希望输入自己的目标岗位、掌握技能、期望地区和薪资偏好，从而得到适合自己的岗位推荐。
5. 作为一名学生，我希望看到推荐岗位与我的技能、地区、经验和薪资偏好的匹配情况，以及还缺少的技能，从而理解推荐结果并安排后续学习。
6. 作为项目使用者，我希望查看 KMeans 岗位聚类结果，了解岗位之间的类型差异和共同特征。
7. 作为项目使用者，我希望查看薪资预测结果及 MAE、 R^2 等评价指标，了解岗位特征与薪资之间的关系。
8. 作为项目维护者，我希望导入公开岗位数据并完成去重、缺失值处理、薪资统一和地区标准化，使后续分析使用统一的数据。
9. 作为项目使用者，当数据为空、筛选无结果或模型未加载时，我希望页面给出明确提示，而不是显示错误结果。

五、技术方案

1. 系统架构

系统采用简单的三层结构：

数据层：公开岗位数据、清洗后数据、数据字典 ↓ 处理层：数据清洗、特征工程、KMeans、随机森林、TF-IDF 推荐 ↓
展示层：Streamlit 页面、岗位检索、统计图表、模型结果、推荐结果

数据层保存原始数据和清洗后的数据。处理层负责数据读取、清洗、特征处理、模型训练和预测。展示层负责接收用户输入，调用处理层函数，并显示表格、图表和推荐解释。前端不直接修改原始数据，也不在页面代码中写死算法结果。

2. 开发平台及语言

- 开发语言：Python；
- 页面形式：Web 页面；
- 主要开发平台：macOS 或 Windows；
- 代码托管和版本管理：Git，远程仓库可使用 GitHub 或学校允许的 Git 仓库。

3. 开发框架及库

- Streamlit：实现页面和交互控件；
- pandas：读取、清洗和统计表格数据；
- NumPy：完成数值处理；
- scikit-learn：完成 KMeans、随机森林回归、TF-IDF 和模型评价；

- Plotly 或 Matplotlib：绘制统计图表；
- SQLite 或 CSV：保存清洗后的数据。第一版优先使用 CSV，避免增加不必要的数据库配置。

4. 数据处理和算法方案

1. 数据采集：以公开岗位数据集为主要来源，在符合网站规则的前提下补充少量公开岗位页面样本，不使用需要登录的页面，不采集个人简历、手机号和账号 Cookie。
2. 数据清洗：处理重复记录、空值、异常薪资、地区名称和经验等级，保留原始数据并生成数据字典。
3. KMeans 聚类：使用岗位类别、技能、经验和薪资等特征划分岗位群组，输出岗位标签和每个群组的特征说明。
4. 随机森林回归：使用岗位类别、地区、经验、教育和技能数量等特征预测薪资，使用 MAE 或 R^2 评价预测结果。
5. TF-IDF 推荐：将学生画像与岗位标题、技能和描述转换为文本向量，使用余弦相似度计算匹配程度，输出 Top-K 岗位、匹配技能和缺失技能。

5. 开发环境、运行环境和项目管理工具

- Python 环境：Python 3.11 及以上；
- 环境管理：uv 或 Miniconda；
- 编辑和调试：Jupyter Notebook、命令行或已有代码编辑器；
- 运行环境：安装 Python 的个人电脑，通过浏览器访问 Streamlit 页面；
- 项目管理：飞书群沟通，AGENTS.md 约束协作和代码范围，《五人任务分工与验收清单》记录任务状态；
- 源代码管理：每人使用个人 Git 分支，main 分支只保留可以运行和演示的版本；
- 代码审查：任务完成后先查看 git diff，由组长或指定验收人检查后合并。

六、团队分工与任务

成员	团队角色	任务列表	主要交付物
卢世豪	算法负责人	负责岗位特征处理、KMeans 岗位聚类、随机森林薪资预测、模型指标和算法模块联调	聚类与回归代码、训练结果、评价指标、结果说明
丁伟哲	算法成员	负责学生画像、TF-IDF 文本向量化、余弦相似度推荐、匹配技能和缺失技能解释	推荐代码、3 个测试画像、Top-K 结果、推荐说明
郑维豪	后端成员	负责公开数据导入、数据采集说明、去重、缺失值处理、薪资和地区标准化、数据字典	数据清洗脚本、清洗后数据、数据字典、处理记录
徐挚凌	后端成员	负责数据读取、岗位筛选、统计函数、模型调用和前后端联调，整理启动说明	数据服务模块、调用说明、联调记录、README 启动步骤
王宇杰	前端成员	负责 Streamlit 页面、岗位检索、数据仪表盘、算法结果、个性化推荐和图表展示	页面代码、交互控件、可视化图表、演示路径

每名成员都要完成一个可以独立说明和演示的任务。任务完成前必须有正常案例、异常或空数据案例、运行步骤和最小验证结果。AI Agent 生成的代码由任务负责人运行、阅读和解释，不能只提交生成结果。

组长由五人协商确定，负责进度同步、README、分支合并和最终演示，不替代其他成员完成算法、后端或前端任务。

七、项目开发计划

项目按 V0.1 至 V1.0 逐步发布，每个版本都以可以运行和演示为准。

版本	时间	主要内容	检查点和交付物
V0.1	第1天	确定题目、数据字段、三类算法和页面草图；用小份数据跑通一次读取和分析	概念验证：能说明数据从哪里来，算法输入输出是什么，页面流程可以演示
V0.2	第2天	完成数据导入、清洗脚本和 Streamlit 页面骨架；三类算法确定函数输入输出	数据清洗结果、数据字典、页面原型、算法接口说明
V0.4	第3天	KMeans、随机森林和 TF-IDF 推荐分别跑通正常案例，完成基本统计图表	三类算法样例结果、基础仪表盘和至少一条完整链路
V0.6	第4天	完成前后端联调、模型评价和异常处理，整理每名成员的阶段成果	中期检查：现场能运行个人负责功能，能说明代码、数据和验证方法
V0.8	第5天	完成推荐解释、页面整理、测试案例、README 和演示流程	验收前版本：从导入数据到页面结果的流程连续可运行
V1.0	第6天	修复阻塞问题，确认数据来源和版本记录，完成演示和材料整理	项目验收：项目可运行、功能可演示、三类算法有输入输出、五人能说明个人贡献

八、可行性分析

1. 技术可行性

项目技术选择控制在课程已经涉及或容易掌握的范围内。Python 负责主要开发，pandas 负责表格数据处理，scikit-learn 提供聚类、回归和文本特征处理工具，Streamlit 可以把 Python 分析结果直接做成网页页面。项目不依赖复杂的前端框架、商业接口或实时登录，因此六天内能够先完成可运行版本，再逐步补充图表和推荐解释。

数据方面，项目使用公开岗位数据集，字段包括岗位标题、描述、地点、经验、技能和薪资等，可以支持清洗、统计、聚类、预测和推荐。对于缺失薪资、空描述和数据量不足的情况，系统保留缺失标记并提示，不通过伪造数据保证结果。

团队按算法、后端和前端分工。算法成员负责模型，后端成员负责数据和模块联调，前端成员负责页面和图表。每个模块都有明确的输入、输出和交付物，减少重复开发。AI Agent 只作为编码辅助工具，生成的代码必须由负责人运行和解释，技术方案仍由团队决定。

2. 团队可行性

团队使用 AGENTS.md 建立统一的项目和代码规范。所有成员开工前先阅读规范和任务清单，只修改自己的任务范围；需求不确定时先记录待确认事项，不让 AI Agent 自行扩大需求。

源代码使用 Git 管理。每名成员从个人分支开发，分支命名为 feat/<姓名>-<任务简称>，每次提交只完成一件事，提交前检查 git diff 和 git status。main 分支只保留能够运行和演示的版本，任务完成后由组长或指定验收人检查，再合并到 main。

团队每天同步一次进展，说明已经完成的内容、当前问题和下一步安排。代码合并前至少检查一次正常案例和一次异常或空数据案例，并确认有运行命令、截图或命令输出。遇到数据获取、依赖安装或模型训练阻塞时，记录已经尝试的办法、具体错误和需要协助的成员，不把失败结果写成完成。

通过任务清单、Git 分支、代码审查和运行证据，能够明确每个人的实际贡献，也能保证项目在时间不足时保留一个可以运行、可以解释的最小版本。